

Benjamin Casarez

benjamin940117@outlook.com · +1 (817) 668-0132 · Fort Worth, TX

PROFESSIONAL SUMMARY

Python developer with 8 years of experience building and maintaining backend applications with Django, Flask, REST APIs, PostgreSQL, and MySQL. Handles Python fundamentals, object-oriented programming, exception paths, database queries, and HTTP behavior without treating framework conventions as magic. Regular work includes unit tests, Git reviews, third-party service integrations, and production debugging across internal tools and customer-facing workflows. Writes implementation notes that help the next engineer find the relevant code, reproduce an issue, and understand known technical debt before making another change.

SKILLS

Languages: Python, Object-oriented programming, Data structures, Exception handling, Functions, SQL

Python Frameworks: Django, Django REST Framework, Flask, FastAPI, Celery, Jinja

Backend APIs: REST APIs, HTTP methods, JSON, API authentication, Third-party integrations, Background services

Databases: PostgreSQL, MySQL, MongoDB, Query design, Database migrations, Data validation

Testing Debugging: Unit testing, Pytest, Mocking, Debugging, Log analysis, Regression testing

Development Tools: Git, GitHub, Pull requests, Docker, Linux, Continuous integration

Engineering Practices: Code reviews, Readable code, Technical documentation, Issue triage, Error handling, Frontend coordination

PROFESSIONAL EXPERIENCE

Python Developer | Altair Engineering | Remote

Jun 2021 - Jul 2026

- Python and Django services handled project configuration and reporting workflows; added validation, clearer exception paths, and REST endpoints used by internal engineering tools.
- PostgreSQL query changes reduced repeated lookups in a report export path by about 18%, while keeping the existing schema and migration process intact.
- An intermittent timeout traced to a retry loop hidden behind a third-party client; corrected the boundary handling and documented the remaining vendor limitation.
- Unit tests with pytest covered serializer errors, permission checks, and malformed requests before review, catching regressions that previously appeared only during manual testing.
- GitHub pull requests included focused code notes and reproduction steps, making small Django changes easier for frontend developers and quality assurance engineers to verify.

Python Application Developer | Torc Robotics | Remote

Jul 2020 - May 2021

- Flask API handlers connected vehicle-support workflows with PostgreSQL records and external service responses, returning consistent JSON errors instead of leaking client-library messages.
- A slow local reproduction depended on stale test fixtures; rebuilt the fixture data, isolated the failing branch, and cut investigation time from roughly 30 minutes.
- Implemented unit tests for HTTP status codes, database failures, and retry behavior, then fixed review findings around duplicated parsing logic and missing exception handling.
- The service kept a few older synchronous calls for compatibility; documented that tradeoff rather than hiding it, with asynchronous replacement left for later maintenance.

Python Engineer | NVIDIA Corporation | Remote

Jul 2018 - Jun 2020

- Python scripts and small Flask services automated hardware test-result collection, converting inconsistent command output into records that engineers could inspect through a simple web endpoint.
- MySQL queries supported inventory and test-history views; added parameterized statements, null handling, and migration notes for changes shared across development environments.
- Git branches, pull requests, and focused reviews kept changes isolated while learning the repository's deployment and logging conventions from more experienced engineers.
- Debugging often meant following one missing field through several adapters; added targeted logs and tests instead of broad logging that would have obscured the actual issue.
- Documentation covered local setup, API examples, and known cleanup work, including a temporary duplicate-data check that still needed a proper database constraint.

EDUCATION

Bachelor of Science in Computer Science (Software Engineering)

Texas State University - San Marcos | San Marcos, TX | Aug 2014 - May 2018